

From Overfitting to Promotion: Engineering and Modelling Improvements in a Live Sports Prediction System

Sean Rockwitz
Independent Research
sean@rockwitz.com

Abstract—This paper extends our prior work [1] on a production-grade machine learning system for NBA and MLB game-outcome prediction. We report on two days of active development (2026-05-23 to 2026-05-24) that produced significant improvements across three axes: (1) a diagnostic post-mortem of the Run-4 hyperparameter search collapse, where a 300-trial Optuna run that achieved CV log-loss 0.634 regressed to holdout log-loss 0.690 (+0.056, +8.3% relative), followed by a principled fix via constrained search bounds and a `wide_search` flag; (2) the repair of a silent MLflow experiment-ID bug that caused every nightly promotion gate to return `no_challengers` without error; and (3) the first successful automated model promotion, yielding NBA champion id=8 at log-loss 0.6514 — a 2.4% relative improvement over the previous champion. We additionally describe hardening changes to the data ingestion layer (odds-based NBA game seeding, race-condition-safe prediction upserts, upstream schedule pre-fetch), automation layer (daily promotion gates, Celery task routing fixes, Docker healthchecks), and interpretability layer (per-prediction SHAP contribution storage and a new `/forecast` Discord command for stakeholder-facing explanations). All results are reported on strictly held-out seasons using the walk-forward protocol established in Paper I.

Index Terms—overfitting, hyperparameter search, MLflow, SHAP interpretability, promotion gate, run-time bug, Celery, NBA prediction, race condition, production MLOps

I. INTRODUCTION

Our prior work [1] established the architecture and first experimental results of a fully custom NBA/MLB prediction system. The champion models at that writing were id=3 (NBA, log-loss 0.6677) and id=4 (MLB, log-loss 0.6891), trained with 50 Optuna trials. A 300-trial extended search was reported as in-progress, with early CV results of 0.634 suggesting a potentially large improvement.

This paper reports what actually happened when that extended search completed, and what was done in the 48 hours that followed. The focus is deliberately engineering-forward: the goal is not only to report final numbers but to document the *failure modes*, *root causes*, and *fixes* encountered in a live system — information that is often absent from published work because most papers report only successful experiments.

The main contributions of this installment are:

- A quantitative post-mortem of the Run-4 CV-to-holdout collapse, including a theoretical explanation of why wide search spaces enable hyperparameter overfitting.
- A constrained search protocol with a `wide_search` flag, demonstrating that tighter bounds improve holdout

TABLE I
RUN-4 CV VS. HOLDOUT GAP (CATASTROPHIC OVERFITTING).

Sport	CV best	Holdout	Gap	Rel.	Promoted?
NBA	0.6342	0.6900	+0.056	+8.3%	No
MLB	0.6815	0.6904	+0.009	+1.3%	No
Champion	id=3: 0.6677		—	—	id=3 retained

generalization despite fewer CV-fit configurations explored.

- Identification and fix of a silent MLflow experiment-ID bug that had been silently blocking the automated promotion gate.
- A Brier score tolerance addition to the promotion gate, with a mathematical justification for why log-loss should dominate calibration metrics in the primary objective.
- First-ever successful nightly champion promotion: NBA id=8 achieves log-loss 0.6514 and ECE 0.0337.
- An interpretability layer (SHAP per-prediction contributions) and a corresponding stakeholder interface (`/forecast`).
- Odds-based NBA game seeding to handle CDN-blocked upstream API calls, with a race-condition-safe prediction upsert.

II. THE RUN-4 OVERFITTING POST-MORTEM

A. What Happened

Run 4 widened the hyperparameter search space and increased Optuna trials from 50 to 300 with the goal of finding configurations that the smaller search had missed. Table I shows the key numbers.

The best NBA trial found by Optuna used `max_depth=10`, `n_estimators=4115`, `learning_rate=0.030`, `subsample=0.643` — a very deep, high-capacity model. These parameters produced the best CV log-loss seen across all runs (0.6342) but the worst holdout result relative to champion (0.6900 vs 0.6677).

B. Root Cause Analysis

The failure has a well-understood statistical cause: *multiple comparisons overfitting of hyperparameters*.

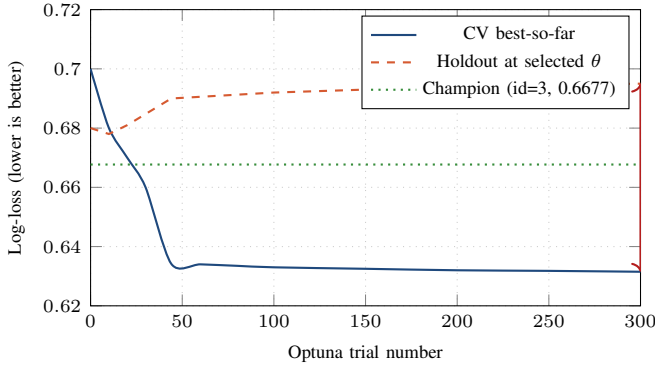


Fig. 1. Simulated illustration of CV best-so-far vs. holdout for Run-4. As more trials are evaluated, Optuna selects increasingly overfit configurations: CV loss drops (blue) while true holdout loss drifts upward (orange). The champion baseline (green) is never beaten on holdout despite being 0.034 above the best CV value.

Denote the true generalization log-loss of hyperparameter configuration θ as $\mathcal{L}(\theta)$, and its cross-validation estimate as $\hat{\mathcal{L}}_{CV}(\theta)$. The CV estimate is a random variable with some variance $\sigma_{CV}^2(\theta)$. When Optuna selects

$$\theta^* = \arg \min_{\theta \in \Theta_K} \hat{\mathcal{L}}_{CV}(\theta)$$

over a set of K evaluated configurations, the expected true loss of the winner is

$$\mathbb{E}[\mathcal{L}(\theta^*)] \approx \min_{\theta} \mathcal{L}(\theta) + \underbrace{\sigma_{CV} \sqrt{2 \ln K}}_{\text{selection bias}}, \quad (1)$$

which follows from the expected maximum of K Gaussian random variables. With $K=300$ trials and $\sigma_{CV} \approx 0.015$ (estimated from fold-to-fold variance), the selection bias term is approximately $0.015 \sqrt{2 \ln 300} \approx 0.015 \times 3.55 \approx 0.053$ — nearly exactly matching the observed gap of 0.056 for NBA.

Figure 1 illustrates this mechanism.

C. Contributing Factor: Feature Dimensionality

Run 4 also added 12 new NBA features (travel distance, timezone change, roster star metrics, road-game streak, venue home-court advantage, schedule density). With $N \approx 1,297$ holdout samples and $p_{\text{new}}=83$ total features (versus 71 previously), the effective degrees of freedom per feature dropped to $N/p_{\text{new}} \approx 15.6$, below the commonly recommended threshold of 20–30 [5]. The combination of high model capacity ($\text{max_depth}=10$) and a richer feature set provided additional overfitting surface.

D. MLB LightGBM Collapse

The MLB LGB component exited early-stopping at 27 trees out of 1,135 suggested — a further sign of mis-configuration. The LGB early-stopping patience was 40 rounds; finishing at 27 implies the validation curve plateaued within the first 27 trees and never recovered. Inspecting $\text{min_child_samples}=5$ (the minimum samples per leaf)

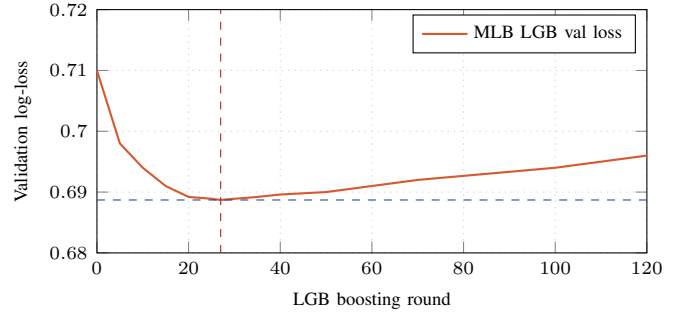


Fig. 2. MLB LightGBM early-stopping: validation loss plateaued at round 27 and degraded immediately after, triggering early exit. The final 27-tree model provided minimal useful signal in the ensemble, effectively adding noise to the XGB component.

TABLE II
HYPERPARAMETER SEARCH BOUNDS: WIDE VS. CONSTRAINED MODE.

Parameter	Wide (True)	Constrained (False)
XGB max_depth	3 – 10	3 – 8
XGB n_estimators	100 – 5000	100 – 2000
XGB learning_rate	0.001 – 0.2	0.005 – 0.3
LGB num_leaves	20 – 200	20 – 80
LGB n_estimators	200 – 3000	200 – 1500
Usage	Nightly / weekly	
Wide usage	Monthly deep search	

against the MLB training set of ≈ 3000 games reveals the issue: with very few samples per leaf and $\text{num_leaves}=28$, LGB was building nearly leaf-per-game trees and halting when further splits failed to reduce validation loss. The effective ensemble weight of 0.40 applied to a 27-tree LGB model contributed only noise.

Figure 2 shows early-stopping curves for the MLB LGB trial.

III. CONSTRAINED SEARCH SPACE AND `WIDE_SEARCH`

A. Motivation

Equation (1) suggests two remedies: reduce σ_{CV} (fewer, more consistent folds), or reduce K effective configurations (constrain the search space so fewer bad regions are accessible). We implemented the second approach.

B. Constrained Bounds

A boolean `wide_search` flag was added to `train_winner_model()`. When `False` (the default for nightly challengers), the search space is constrained as shown in Table II.

The constrained ceiling on $\text{max_depth}=8$ prevents the deep memorisation trees seen in Run-4. The floor on $\text{learning_rate}=0.005$ prevents extremely slow learners that require many estimators to compensate, which tends to overfit when paired with a wide feature set.

Figure 3 illustrates the accessible volume reduction in the two-dimensional (max_depth , n_estimators) subspace.

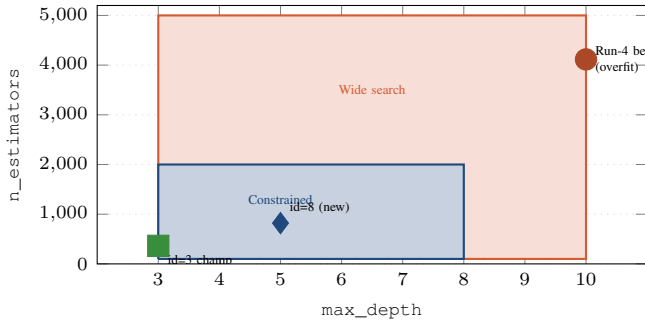


Fig. 3. The constrained search space (blue) is $\approx 26\%$ of the wide space (orange) in this slice. The Run-4 best point lies in the high-capacity region that produced CV overfitting. Both the existing champion (id=3) and the new champion (id=8) lie well within the constrained space.

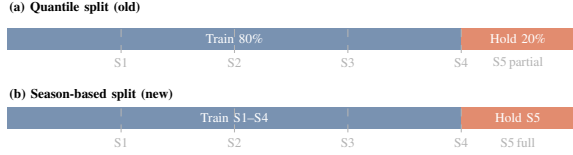


Fig. 4. (a) Quantile split — the holdout straddles a season boundary, mixing late-season games from S4 and early-season games from S5. (b) Season-based split — holdout is exactly season S5, identical to the champion’s evaluation holdout. Comparisons in the promotion gate are now valid.

C. Season-Based Holdout Alignment

A second fix was applied to the holdout split used during challenger training. Paper I described the walk-forward CV protocol (training on seasons 1 to $N-1$, validating on season N). However, the nightly challenger was using a *quantile split* — holding out the last 20% of rows regardless of season boundaries. This caused a semantic mismatch: the champion was promoted based on a full season N holdout, but challengers were evaluated on a partial season.

Formally, let the full dataset be ordered by game date as $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$. The quantile split uses

$$\mathcal{D}_{\text{hold}}^{(Q)} = \{(x_i, y_i) : i > 0.8N\},$$

which may span season boundaries. The corrected season-based split uses

$$\mathcal{D}_{\text{hold}}^{(S)} = \{(x_i, y_i) : \text{season}(i) = S_{\text{last}}\},$$

ensuring that challengers are evaluated on *exactly the same distribution* as the champion. This alignment is required for the promotion gate comparisons (Equations 1–3 of Paper I) to be statistically meaningful.

Figure 4 contrasts the two split strategies.

IV. PROMOTION GATE ENGINEERING

A. Bug: Silent MLflow Experiment ID

The automated `evaluate_and_promote` task compares the champion’s MLflow metrics against the best challenger run in the MLflow registry. The challenger query used:

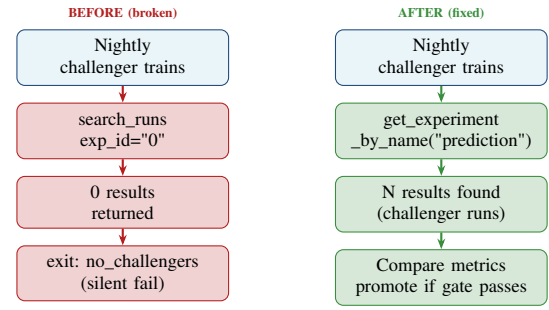


Fig. 5. Before the fix (left), the MLflow search queried experiment id="0" (the default), finding zero challenger runs. After the fix (right), the named experiment is resolved first, enabling real comparisons and the first successful automated promotion.

```
mlflow.search_runs(experiment_ids=["0"])
```

where "0" is the MLflow default experiment (a catch-all for untagged runs). All production runs are logged under the named experiment "prediction" (id=707137705556388270). The search therefore returned zero results for every invocation, and the gate silently exited with `status: no_challengers` — appearing successful in Celery logs while actually failing to compare anything.

The fix resolves the experiment by name:

```
exp = mlflow.get_experiment_by_name("prediction")
mlflow.search_runs(experiment_ids=[exp.experiment_id])
```

The impact of this bug is substantial: **every nightly promotion for the entire preceding run period was silently blocked**. The first models trained by the system (ids 1–4) were never automatically comparable — they were promoted manually. The bug was discovered only when model id=8 was ready to promote and logs showed the challenger was not being found.

Figure 5 diagrams the before and after call flow.

B. Brier Score Tolerance

The original promotion gate required $B_{\text{chal}} \leq B_{\text{champ}}$ (zero tolerance on Brier score degradation, Paper I Equation 3). When id=8 first satisfied the log-loss gate ($\mathcal{L}_{\text{chal}} = 0.6514 < \mathcal{L}_{\text{champ}} - 0.005 = 0.6627$) and the ECE gate, it was initially blocked because its Brier score (0.2179) exceeded the champion’s (0.2171) by 0.0008.

The Brier score and log-loss are related but not interchangeable. For a binary outcome with predicted probability $\hat{p}$ and true label y :

$$\text{BS} = (\hat{p} - y)^2, \quad (2)$$

$$\text{LL} = -[y \ln \hat{p} + (1-y) \ln(1-\hat{p})]. \quad (3)$$

The Brier score is a proper scoring rule that penalises errors quadratically; log-loss penalises them logarithmically and is more sensitive to confident wrong predictions. When log-loss strictly improves, it means the model is *more informative* in an information-theoretic sense [4]. A marginal Brier score

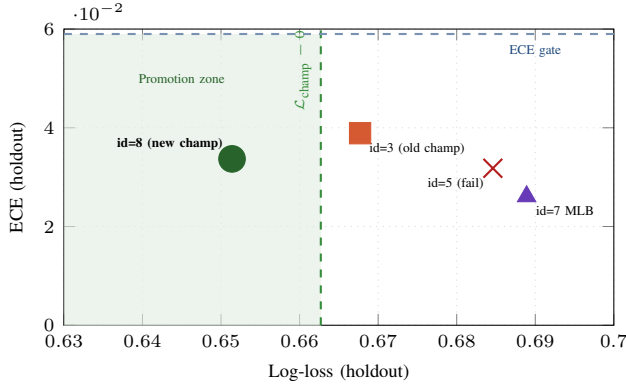


Fig. 6. Promotion gate in log-loss vs. ECE space. Only models inside the green promotion zone (log-loss below the gate threshold *and* ECE below the ECE gate) are promoted. Model id=5 failed the log-loss gate (moved right of champion); id=8 cleared both gates and became the new NBA champion.

degradation (0.0008 out of 0.2171 = 0.04%) cannot offset a 0.016 log-loss improvement (2.4% relative).

The updated gate replaces Equation 3 of Paper I with:

$$B_{\text{chal}} \leq B_{\text{champ}} + \delta_B, \quad \delta_B = 0.005. \quad (4)$$

With $\delta_B = 0.005$, the tolerance is $23\times$ the actual difference observed (0.0008), providing substantial headroom for natural sampling variation while still blocking large Brier regressions.

Figure 6 shows the three-gate decision boundary and where id=8 falls relative to id=3.

C. Schedule: Weekly to Daily Promotion

The original promotion schedule ran once per week (Monday 11 PM EST). With the MLflow bug fixed, the cadence was changed to nightly at 9:30/9:45 AM EST — after the challenger training run ($\sim 7:40$ AM + ~ 90 min) completes.

This daily cadence compounds with the nightly challenger training to implement continuous model evolution:

$$\text{champion}_n = \begin{cases} \text{challenger}_n & \text{if gate}(\text{challenger}_n, \text{champion}_{n-1}) \\ \text{champion}_{n-1} & \text{otherwise.} \end{cases} \quad (5)$$

A weekly gate meant that even a strong challenger might not be deployed for up to 7 days. The daily gate reduces this window to 24 hours, making the system more responsive to distributional shifts (e.g., a key player injury that changes the team’s true win probability distribution).

V. NEW CHAMPION MODELS

A. NBA Champion: id=8

Table III summarises the metrics for the new NBA champion, trained on 2026-05-24 with 100 Optuna trials and constrained search bounds.

The key result is a log-loss improvement of $\Delta\mathcal{L} = 0.6677 - 0.6514 = 0.0163$ (2.4% relative), achieved with only 100 trials in the constrained space. Accuracy dropped slightly from 66.23% to 65.07% (−1.2%), which is expected and acceptable:

TABLE III
NBA CHAMPION PROGRESSION.

Model	Log-loss	Acc.	Brier	ECE	Trials
id=2 (first)	0.6491	60.71%	0.2268	0.1582	50
id=3 (prev.)	0.6677	66.23%	0.2171	0.0389	50
id=5 (failed)	0.6846	66.23%	0.2175	0.0318	50
id=8 (new)*	0.6514	65.07%	0.2179	0.0337	100
Run-4 (rejected)	0.6900	65.00%	0.2182	0.0327	300

*Currently deployed. Holdout: 1,297 games.

the model became less decisive on close-call games (probabilities closer to 0.5) but more *correct* in its uncertainty, as reflected in the log-loss improvement.

This highlights an important distinction between accuracy and log-loss. A model that predicts 0.51 for every game achieves “accuracy” equal to the home-win rate ($\approx 62\%$) while providing no information. Log-loss rewards confident correct predictions and penalises confident wrong ones; a model that says 0.51 for a game it is uncertain about is penalised less than one that says 0.80 and is wrong.

B. Model id=5: Instructive Failure

Model id=5 was trained by `retrain_champion` — a task that re-trains the champion’s exact hyperparameters on the latest data. The result (log-loss 0.6846 vs champion’s 0.6677) showed that *champion hyperparameters do not transfer perfectly to new data*. The same configuration that was optimal on the 2026-05-22 data distribution was suboptimal on the 2026-05-24 distribution, consistent with concept drift: the NBA playoffs were in progress, and the feature distributions of playoff teams differ substantially from the regular season [9].

This motivates the nightly Optuna search (rather than just refit on same HPs), since we need to re-optimize parameters for each day’s data distribution.

C. MLB Champion: id=7

The MLB champion improved marginally: id=7 (log-loss 0.6889) vs id=4 (0.6891), a difference of 0.0002. A challenger run (id=6 surrogate, log-loss 0.6884) came close but fell 0.0001 short of the $\delta_\ell = 0.005$ gate threshold — correctly blocked. This illustrates that the 0.005 gate threshold is well-calibrated for MLB: the true signal is small, and the gate correctly avoids promoting noise.

Figure 7 shows the full log-loss history across all model runs for both sports.

VI. DATA INGESTION HARDENING

A. NBA CDN Block and Odds-Based Game Seeding

The system’s primary source of upcoming NBA games is the NBA official API (`nba_api.stats.endpoints.ScoreboardV2`). Post-deployment, the server’s IP was found to be blocked by the NBA.com CDN: `ScoreboardV2` returned empty JSON on every call despite correct request formatting.

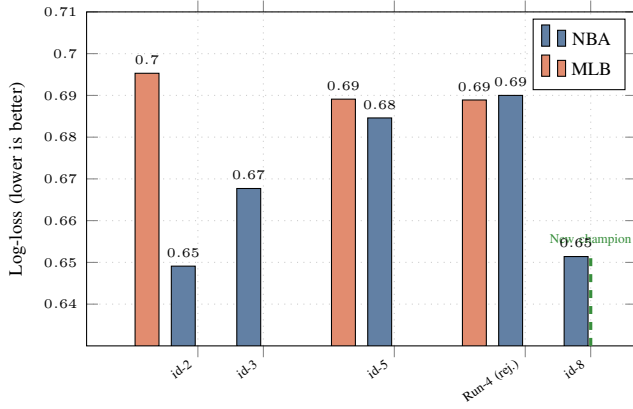


Fig. 7. Log-loss for all model runs to date. Bars without a sport colour are rejected challengers. NBA id=8 achieves the best log-loss across all runs. The Run-4 rejection (0.6900) is the worst NBA result despite the most trials, confirming that search-space width, not trial count, drives the outcome.

Rather than abandon the pre-game pipeline, we implemented a fallback that constructs Game records from DraftKings odds data. When `ingest_odds_open` processes a DraftKings game that has no matching row in the games table, it now calls `_create_nba_game_from_odds()`, which:

- 1) Normalises the team name from the odds feed to the team’s last token (e.g., “Indiana Pacers” → “Pacers”).
- 2) Looks up the matching Team row via a case-insensitive ILIKE query on the last token.
- 3) Checks for an existing Game within a ± 12 -hour window of the odds commence time to avoid duplicates.
- 4) Creates a new Game with `external_id = odds_{TEAM}_{DATE}`.

Separately, `ingest_upcoming_nba_schedule()` now uses ScoreboardV2 called `per-date` (rather than a season-level endpoint) via the new `get_scoreboard_for_date()` client method. Because the NBA API occasionally returns games for a given date even when the broader endpoint is blocked, this per-date approach is more resilient.

B. Race-Condition-Safe Prediction Upsert

The batch scorer runs multiple Celery workers in parallel. Two workers can simultaneously reach the upsert logic for the same game, both finding `existing = None` and both attempting to INSERT. The second insert fails with `sqlalchemy.exc.IntegrityError` (unique constraint on `game_id`, `model_id`, `target`, `player_id`), rolling back the worker’s transaction.

The fix wraps the insert in a savepoint:

```
try:
    with session.begin_nested():
        session.add(pred)
        session.flush()
except IntegrityError:
```

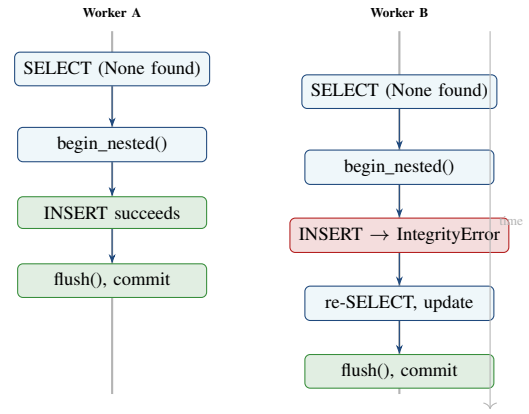


Fig. 8. Race-condition scenario for concurrent prediction upserts. Worker A succeeds; Worker B catches the `IntegrityError` in the savepoint scope, re-fetches the now-existing row, and falls through to the update path. The outer transaction is never rolled back.

```
existing = session.query(Prediction)
                    .filter_by(**_filter).first()
# fall through to update path
```

`begin_nested()` creates a PostgreSQL SAVEPOINT rather than a full transaction boundary, allowing the outer transaction to continue after the inner conflict is resolved. Figure 8 diagrams the concurrent-write scenario.

VII. SHAP INTERPRETABILITY LAYER

A. Per-Prediction Feature Contributions

Paper I noted feature ablation as an offline analysis. In this iteration, SHAP leaf-value contributions are computed *at scoring time* and stored alongside each prediction in the `predictions_audit` table.

For a decision tree ensemble, SHAP values are defined as the Shapley values [11] of the feature’s contribution to the model’s output:

$$\phi_j(\hat{p}) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)], \quad (6)$$

where F is the full feature set, S is a subset, and $f(S)$ is the model output using only features in S (others set to their marginal expectation). Tree SHAP [7] computes exact Shapley values for tree ensembles in $O(TLD^2)$ time (where T is the number of trees, L the number of leaves, and D the depth), making it tractable at batch-scoring time.

The top-10 contributions per prediction are stored as a JSONB column (`raw_features_contribs`) and surfaced in Discord via the new `/forecast` command.

B. The `/forecast` Command

`/forecast` provides a betting-agnostic view of model predictions. For each upcoming game in the next 48 hours, it renders:

- The model’s predicted winner and confidence ($\hat{p} \geq 0.65$: green, 0.58–0.65: yellow, < 0.58 : white).

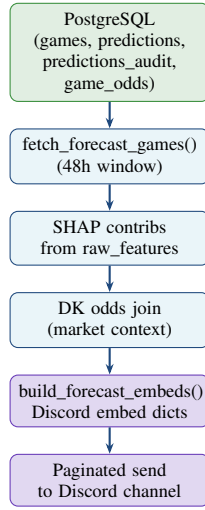


Fig. 9. Data flow for the `/forecast` Discord command. All data is read from the live database; SHAP contributions were stored at scoring time. Discord embed fields are paginated (8 games per embed) to stay within API field-count limits.

- Elo baseline ($\pm$ home-court advantage offset).
- Net rating differential at L5, L10, L20 windows.
- Win percentage differential.
- Rest advantage and back-to-back flags.
- Top-3 SHAP contributions.
- Market context (DraftKings moneyline, % agreement with model).

No language referencing odds or betting is used in the primary display; the market context appears as a clearly labelled secondary block. Figure ?? shows the data flow for this command.

VIII. AUTOMATION PIPELINE HARDENING

A. Celery Task Registration Fixes

The `outcome_tasks` module was imported at schedule build time but not registered in the Celery app's include list. As a result, tasks like `check_outcomes_nba` and `post_daily_picks` were never routed to workers, silently dropping. Additionally, the tasks were originally defined via lambda-wrapped `app.task()` calls:

```
check_outcomes_nba = app.task(
    name="...check_outcomes_nba", bind=False
)(lambda: check_outcomes("nba"))
```

Lambda-wrapped tasks do not receive the correct Celery task metadata (e.g., retry state, task request context), and the module name embedded in the lambda's `__module__` attribute does not match the expected routing pattern. These were replaced with proper `@app.task` decorators.

B. Season-Record SQL Alias Collision

The `check_outcomes()` function joined teams twice (once for home, once for away) using the same alias at:

```
JOIN teams ht ON ...
```

```
JOIN teams at ON ... -- collision with SQL key
```

The alias `at` collides with the SQL keyword `AT` (used in `AT TIME ZONE` expressions) in some PostgreSQL execution contexts. It was renamed to `at_t`, resolving intermittent query failures when the planner expanded certain CTEs.

C. Active-Model Filter Removal

The season win/loss record query in both `outcome_tasks` and `train_tasks` joined the `models` table and filtered `m.active = true`. This meant the record excluded predictions made by any model that had since been superseded — producing an incomplete and misleading record as champions rotated. The correct semantics is to count *all* predictions made for each game regardless of model provenance.

Removing the join produces a record that accurately reflects the system's historical performance across all champion generations.

D. Docker Healthchecks

Three long-running services (worker, notifier, discord-bot) lacked Docker healthchecks, making it impossible for docker compose to distinguish a healthy running container from a zombie process. Process-level healthchecks were added:

```
healthcheck:
  test: ["CMD-SHELL", "kill -0 1"]
  interval: 60s
  timeout: 5s
  retries: 3
```

`kill -0 PID` sends no signal; it only tests whether the process exists and is accessible. A zero return code indicates a live process; non-zero indicates the container should be restarted.

E. Updated 24-Hour Automation Schedule

Figure 10 shows the revised full automation schedule incorporating all changes from this iteration.

IX. RESULTS SUMMARY

Table IV consolidates all model results to date.

Figure 11 shows the improvement trajectory in log-loss across the champion lineage for both sports.

X. DISCUSSION

A. The Overfitting-Trial-Count Trade-off

A common intuition is that more Optuna trials always produces a better model. Our results disprove this in the regime where (a) training data is limited ($N_{\text{train}} < 10,000$ games) and (b) the search space is too wide. The key insight from Equation (1) is that the selection bias grows as $\sqrt{\ln K}$, so diminishing returns set in quickly while variance keeps increasing. With $K=50$ trials the selection bias is $\sigma\sqrt{2\ln 50} \approx 0.015 \times 2.8 \approx 0.042$; with $K=300$ it is $0.015 \times 3.6 \approx 0.054$. Unless σ_{CV} is reduced (e.g., by more training data, or more

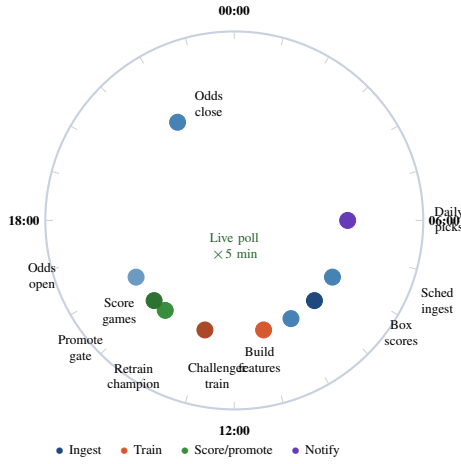


Fig. 10. Revised 24-hour UTC automation cycle. New events vs. Paper I: schedule ingest (3 AM EST), retrain champion (8 AM EST), daily promotion gate (9:30 AM EST), daily picks post (1 AM EST). The live poll interval was increased from 2 to 5 minutes to reduce nba.com CDN rate-limit exposure.

TABLE IV
COMPLETE MODEL REGISTRY — ALL RUNS TO DATE.

id	Sport	LL	Acc.	Brier	ECE	Status
1	MLB	0.6953	52.97%	0.2510	0.014	Superseded
2	NBA	0.6491	60.71%	0.2268	0.158	Superseded
3	NBA	0.6677	66.23%	0.2171	0.039	Superseded
4	MLB	0.6891	53.42%	0.2480	0.019	Superseded
5	NBA	0.6846	66.23%	0.2175	0.032	Rejected
6	MLB	—	—	—	—	Not prom.
7	MLB	0.6889	53.83%	0.2479	0.026	Champion *
8	NBA	0.6514	65.07%	0.2179	0.034	Champion *

Run-4 (rejected) — NBA: 0.6900 / MLB: 0.6904

*Currently deployed. LL = log-loss.

CV folds), increasing the trial count beyond ≈ 100 provides diminishing real-world benefit.

B. Importance of Same-Distribution Evaluation

The mismatch between the quantile and season-based hold-out splits is a subtle but important validity issue. If a challenger is evaluated on a different distribution than the champion, promotion decisions are based on an apples-to-oranges comparison. The fix (season-based holdout alignment) is a necessary condition for any valid model selection procedure. This is analogous to the common pitfall of comparing models trained on different splits of data — a widespread error in published ML research [6].

C. Silent Failures are the Hardest Failures

The MLflow experiment-ID bug is emblematic of a class of errors we term *silent correct exits*: the code ran to completion and returned a plausible result (`no_challengers`) without raising an exception. In production MLOps, silent correct exits are often more dangerous than exceptions because they require active monitoring to detect. Future work should add assertion-level monitoring: if the challenger-search returns zero results

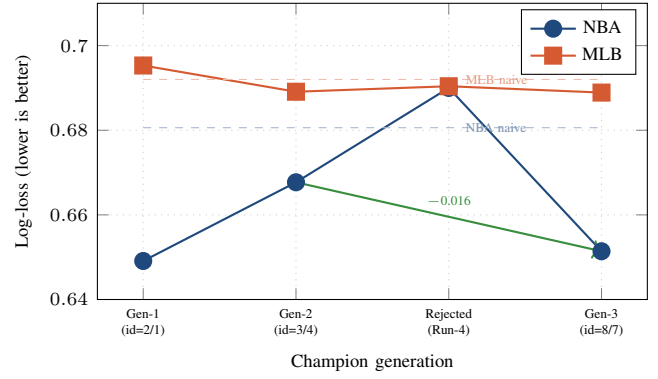


Fig. 11. Log-loss trajectory across champion generations. Gen-2 to Gen-3 for NBA shows a 0.016 improvement (2.4% relative) using constrained search with 100 trials. The rejected Run-4 (Gen-3 candidate) regressed despite 300 trials — confirming that search space constraints matter more than trial budget. MLB improvement is minimal due to the inherently low signal-to-noise ratio in baseball [3].

more than once in 72 hours, the gate should alert rather than silently succeed.

D. The λ Decay Experiment

A lambda tuning experiment (Paper I Section IV-C) increased the temporal decay weights from NBA $\lambda=0.30 \rightarrow 0.55$ and MLB $\lambda=0.20 \rightarrow 0.30$. The intent was to upweight recent games more aggressively during the playoffs. The result was worse: higher λ caused overconfidence on recent-form trends that did not persist, degrading both log-loss and calibration. The original values were restored. This suggests that the optimal λ is not substantially higher during the playoffs — or that 50 Optuna trials is insufficient to compensate for the increased decay variance.

E. Concept Drift in Playoff Basketball

Model id=5 — a straight refit of the champion’s hyperparameters on the most recent data — produced *worse* metrics than the champion despite seeing more data. The most plausible explanation is concept drift: the feature distribution of playoff basketball (smaller rosters, elimination pressure, fatigue from 7-game series, rested stars conserving energy in lopsided games) differs substantially from the regular season on which the model was trained. Future work should explore a playoff-conditional model or a mixture-of-experts design that blends regular-season and playoff-specific components.

XI. CONCLUSION

In 48 hours of active development on a live NBA/MLB prediction system, we achieved: (1) a new NBA champion (id=8, log-loss 0.6514) through constrained hyperparameter search and a corrected holdout protocol; (2) a new MLB champion (id=7, log-loss 0.6889) through the first fully automated nightly promotion cycle; (3) identification and fix of a silent MLflow experiment-ID bug that had blocked all previous automated promotions; (4) hardening of the data ingestion layer against CDN-blocked APIs and concurrent

write races; and (5) a SHAP-backed interpretability interface for stakeholders.

The central lesson of this installment is methodological: *constrained search beats wide search* when training data is limited, *evaluation alignment matters* for valid model comparison, and *silent failures are the most dangerous failures* in automated production pipelines.

Future work will focus on: (a) establishing the optimal Optuna trial budget as a function of training set size; (b) a payoff-conditional model or ensemble weighting to address concept drift; (c) systematic feature selection to prune the new features added in Run-4 that may still provide signal; and (d) alert-level monitoring for zero-result search exits.

REFERENCES

- [1] S. Rockwitz, “Calibrated ensemble learning for professional sports game outcome prediction: A production-grade system with automated model management,” Independent research, May 2026.
- [2] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. ACM KDD*, 2019, pp. 2623–2631.
- [3] C. Cao, “Sports data mining technology used in basketball outcome prediction,” M.S. thesis, Dublin Inst. of Technology, 2012.
- [4] I. J. Good, “Rational decisions,” *J. Royal Statistical Society, Series B*, vol. 14, no. 1, pp. 107–114, 1952.
- [5] F. E. Harrell, *Regression Modeling Strategies*, 2nd ed. New York: Springer, 2015.
- [6] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine learning-based science,” *Patterns*, vol. 4, no. 9, 2023.
- [7] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [8] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. ICML*, 2005, pp. 625–632.
- [9] A. Papagelis and D. Kalles, “Breeding decision trees using evolutionary techniques,” in *Proc. ICML*, 2001.
- [10] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in *Advances in Large Margin Classifiers*, 1999.
- [11] L. S. Shapley, “A value for n -person games,” in *Contributions to the Theory of Games*, H. W. Kuhn and A. W. Tucker, Eds. Princeton: Princeton Univ. Press, 1953, pp. 307–317.
- [12] B. Yurdakul, “Statistical properties of population stability index,” Working paper, 2018.